

Compiling ZDoom/GZDoom on Windows

To build ZDoom on Windows, you have a few options:

- Use Visual Studio Community
- Use MinGW

Required downloads for both Visual C++ and MinGW

- [CMake](#) Version 2.8.7 or later
- The ZDoom source code, either from an [official release](#) or from the [Git repository](#).

Optional downloads for both Visual C++ and MinGW

Currently OpenAL is only supported when building with CMake. You only need these if you want to statically link to these libraries or compile GZDoom 2.4.0 or older with OpenAL support. The project contains all needed headers to dynamically link to them by default and all that needs to be provided are the DLLs from the binary download.

- [OPENAL SDK](#)
 - libsndfile [32-bit](#) or [64-bit](#)
 - libmpg123 [32-bit](#) or [64-bit](#)

Visual C++

The easiest way to compile ZDoom on Windows is to use CMake to generate Visual Studio projects.

Note: Visual Studio 2017 or newer is required for building the latest GZDoom. The last stable ZDoom release 2.8.1 can be compiled with 2005-2013.

Required Downloads for Visual C++

- [Visual Studio Community Edition](#)

The following downloads are needed when building with Windows XP compatibility:

- [Windows Software Development Kit \(SDK\) for Windows 8.1](#)
- [DirectX Software Development Kit \(February 2010\)](#)

Download and install each of the above. For Visual Studio make sure that you select the Visual C++ and XP support components in the custom options in the installer as they are not installed by default.

Preparing library

Note: You only need to do this step if you want to build the 2.4.0 release or older with OpenAL support!

libmpg123 doesn't come with a library file, so you will need to build one.

1. Open the developer command prompt

- Open the start menu and browse to the "Visual Studio" folder.
- Open the "Visual Studio Tools" folder.
- Run the "Visual Studio command prompt" or "developer command prompt" depending on your Visual Studio installation.

2. Browse to libmpg123 directory

- Manually type the directory where you installed mpg123 into the command prompt. For example:

```
cd C:\Program Files (x86)\mpg123
```

3. Rename export definition file

- Due to an error in one of the file names included with mpg123, ZDoom will fail to load on startup. To fix this, rename libmpg123-0.dll.def to libmpg123-0.def
- You can do this in the command prompt or in windows explorer. For the former:

```
rename libmpg123-0.dll.def libmpg123-0.def
```

4. Build library

- 32-bit

```
lib /DEF:libmpg123-0.def /OUT:libmpg123-0.lib
```

- 64-bit

```
lib /MACHINE:X64 /DEF:libmpg123-0.def /OUT:libmpg123-0.lib
```

If done correctly, libmpg123-0.lib and libmpg123-0.exp should have been created in the directory.

Compiling LZDoom07:

1. Find a legacy cmake versions (3.31 or a bit lower)

2. Run cmake-gui.exe

- In "Where is the source code:" browse to the zdoom folder from above
- In "Where to build the binaries:" create a build directory in the zdoom folder above
- Click configure
- Select "Visual Studio <version>" or "Visual Studio <version> Win64" for the generator based on the Visual Studio version you have and whether you want a 32 bit or 64 bit build. (For example, "Visual Studio 14 2017 Win64" if you're using Visual Studio 2017 and want a 64-bit build)
- **If you want to build with Windows XP compatibility, enter "v140_xp" (without the quotes) in the optional toolset box**
- Click Finish

2. After cmake configure finishes, it will likely fail because it couldn't find some things:

If you are building 2.4.0 or older with FMOD support:

- set FMOD_INCLUDE_DIR to 'api/inc' folder inside folder where you installed fmod earlier (e.g. C:\Program Files (x86)\FMOD SoundSystem\FMOD Programmers API Windows\api\inc)
- set FMOD_LIBRARY to 'api/lib/fmodex_vc.lib' inside fmod folder (e.g. C:\Program Files (x86)\FMOD SoundSystem\FMOD Programmers API Windows\api\lib\fmodex_vc.lib). If you are compiling for 64-bit systems set FMOD_LIBRARY to fmodex64_vc.lib instead.

3. Add the OpenAL requirements. This is only necessary for GZDoom 2.4.0 and older. The following is an example:

```
OPENAL_INCLUDE_DIR C:/Program Files (x86)/OpenAL 1.1 SDK/include
OPENAL_LIBRARY C:/Program Files (x86)/OpenAL 1.1 SDK/libs/Win64/OpenAL32.lib
SNDFILE_INCLUDE_DIR C:/Program Files/Mega-Nerd/libsndfile/include
SNDFILE_LIBRARY C:/Program Files/Mega-Nerd/libsndfile/lib/libsndfile-1.lib
MPG123_INCLUDE_DIR C:/Program Files (x86)/mpg123
MPG123_LIBRARIES C:/Program Files (x86)/mpg123/libmpg123-0.lib
```

Be sure to use the right library from the OpenAL SDK. Note that for version 3.0.0 and above this is not needed anymore.

4. Click 'Configure'

- If this fails with the message 'Could not find DirectX 9 libraries', you likely have an older DirectX SDK or installed the SDK into a non-standard location. To workaround this, you'll likely have to set the following cmake variables manually in the cmake GUI dialog (following is only an example):

```
D3D_INCLUDE_DIR C:/Program Files (x86)/Microsoft DirectX SDK (June 2010)/Include
DX_dinput8_LIBRARY C:/Program Files (x86)/Microsoft DirectX SDK (June
2010)/Lib/x86/dinput8.lib
DX_dxguid_LIBRARY C:/Program Files (x86)/Microsoft DirectX SDK (June
2010)/Lib/x86/dxguid.lib
```

This step is only required when building with Windows XP compatibility. The modern toolsets for Windows Vista and up will use an SDK version that includes these headers by default.

- You are also likely to see the following reported:

```
Could NOT find BZip2 (missing:  BZIP2_LIBRARIES BZIP2_INCLUDE_DIR)
Could NOT find JPEG (missing:  JPEG_LIBRARY JPEG_INCLUDE_DIR)
Could NOT find ZLIB (missing:  ZLIB_LIBRARY ZLIB_INCLUDE_DIR)
Could NOT find GME (missing:  GME_LIBRARIES GME_INCLUDE_DIR)
Using internal zlib
Using internal jpeg library
Using internal bzip2 library
Using internal gme library
Could NOT find PkgConfig (missing:  PKG_CONFIG_EXECUTABLE)
```

These are merely informational notices. The "missing" files are not required (internal versions are used for the first four and PkgConfig is not required). A fully functional compile will still be made.

5. Click 'Generate'. After this, cmake should have successfully created 'zdoom.sln', and 'ALL_BUILD.vcxproj.user' files in the directory given for "Where to build the binaries:" (look at timestamp on the files as a sanity check). If not, something's gone wrong.

6. Close cmake

7. Open 'zdoom.sln' file in the build folder created earlier (double-clicking or within VS makes no difference)

8. Click Build -> Build Solution (NOTE: The build will fail if you have the Debug or Release folder inside the zdoom folder open due to one of the post-build steps.)

9. When the build successfully completes (there should be no failures):

- copy the 'fmodex.dll' or 'fmodex64.dll' to the zdoom/Debug folder (e.g. cp C:\Program Files (x86)\FMOD SoundSystem\FMOD Programmers API Windows\api\fmodex.dll C:\Users\Shawn\Devel\zdoom\Debug)
- copy 'openal32.dll' to zdoom/Debug folder.
- copy 'libmpg123-0.dll' to zdoom/Debug folder.
- copy 'libsndfile-1.dll' to zdoom/Debug folder.

10. Now zdoom.exe can be opened from the zdoom/Debug folder, or from within Visual Studio using Debug -> Start Debugging (F5) when the solution is selected. Assuming it found your copy of installed Doom (which if you've installed The Ultimate Doom from Steam, it will automatically), the game should launch after a few moments and go to the main menu. When debugging, ensure that 'zdoom' is marked as the startup project.

MinGW

Downloads for MinGW

- [MinGW-w64](#) 32 or 64 bit version.

32 bit build: i686-8.1.0-release-posix-sjlj-rt_v6-rev0.7z

64 bit build: x86_64-8.1.0-release-posix-seh-rt_v6-rev0.7z

- [Code::Blocks IDE](#) (optional). Download the installer without TDM-GCC (codeblocks-17.12-setup.exe).

Preparing for compiling

- Extract i686-8.1.0-release-posix-sjlj-rt_v6-rev0 to any folder. (e. g. C:/DEV)

You need to add the path to the MinGW binaries to the system environment vars (Control panel->system->advanced system settings->environment vars). Add ';C:\DEV\mingw32\bin' to the Path variable.

- Unpack the ZDoom source code wherever you like.
- Run cmake-gui (we'll be using 3.9.6). You can find it in the start menu after installing [CMake](#).
 - In the box labeled "Where is the source code:" point it to the directory where you put the source code. (e.g. C:/DEV/ZDoom)
 - In the box labeled "Where to build the binaries:" create a subdirectory in the path to the source code to contain the build files (e.g. C:/DEV/ZDoom/release).
 - Click the Configure button at the bottom of the window. If the build directory does not exist, it will ask to create it. Let it.
 - Select CodeBlocks MinGW Makefiles as the project generator (or just MinGW makefiles if you don't plan to use the IDE).
 - Then specify manually the native compilers (g++.exe and gcc.exe) in 'C:/DEV/mingw32/bin'.
 - Check the configuration options to make sure they are to your liking. In particular, pay attention to:

- CMAKE_BUILD_TYPE: This defaults to Debug. You probably want to change it to Release.
- Set DX_dinput8_LIBRARY and DX_dxguid_LIBRARY to 'C:/DEV/mingw32/i686-w64-mingw32/lib/libdinput.a' and 'libdxguid.a'.
- If you want XInput support you'll need the full DX SDK. Set XINPUT_INCLUDE_DIR to 'C:/Program Files/Microsoft DirectX SDK (November 2008)/Include'.
- It's recommended to tick the CMAKE_VERBOSE_MAKEFILE option.
- If you made any changes, press Configure again.
- Now press Generate. This will create a Makefile in the build directory.

Compiling

- Open a Command Prompt window. You can either use Start -> Run... and type "cmd" or you can find it in Start -> All Programs -> Accessories -> Command Prompt.
- It's recommended to increase buffer size for the command prompt to at least 500.
- C:\DEV\mingw32\bin should be in your path (else cmake-gui would have failed). To set this for just the current command prompt, use the command

```
set PATH=C:\DEV\mingw32\bin;%PATH%
```

To make this automatic for all command prompts, you need to edit the Environment Variables on the Advanced tab of the System Properties control panel.

- Change to the directory you chose as the build directory. e.g. For the example above

```
cd C:/DEV/ZDoom\release
```

- Compile the code from inside the build directory

```
mingw32-make
```

You can also use a multithreaded make to speed up compilation:

```
mingw32-make -j2
```

Just replace the 2 with however many threads you want to use.

- You may want to build from inside the IDE instead for debugging, just double click on the ZDoom.cdb project.
- This will leave a `zdoom.exe` in the build directory.

Additional Notes

- The current source should compile and run but there are a few problems (reported as bugs). MinGW links against an ancient MS C runtime and to extend its functionality there's a MinGW CRT but it's very buggy so expect some crashes on exit.
- Also as of this writing the program cannot be debugged anymore on certain machines (game hangs on startup).